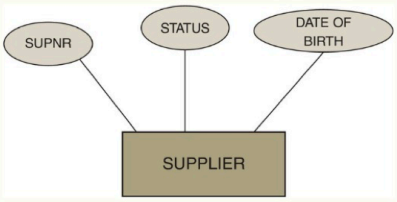
02 ER-Models

Chapter 3.0 to 3.3 - Conceptual Data Modeling

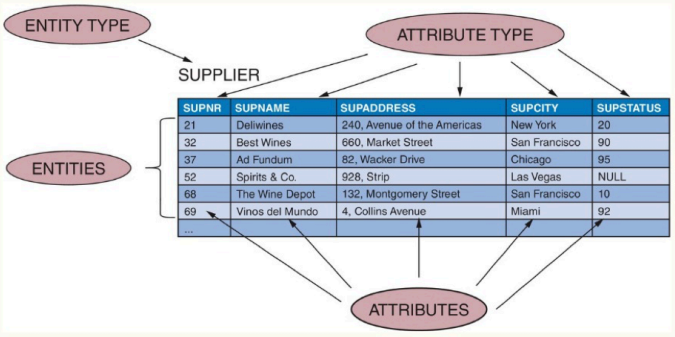
1. Entity-Relationship (ER) Model Basics

Entities:

- Represent real-world objects (e.g., *Student*, *Car*, *Employee*).
- Can be classified as:
  - **Strong Entities:** Exist independently.
  - **Weak Entities:** Depend on other entities for identification.




```sql
CREATE TABLE Supplier (
   SupNr INT NOT NULL,
   SupStatus INT,
   SupCity VARCHAR(50) NOT NULL,
   ...
);
```

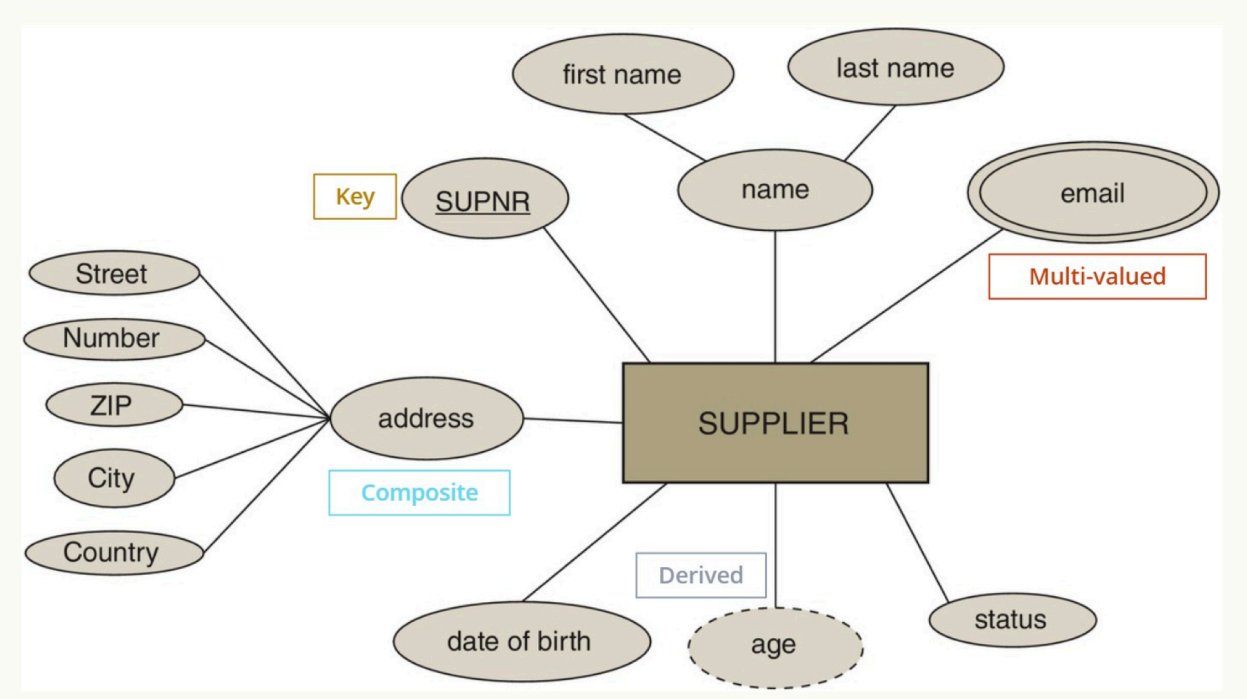


SUPNR	SUPNAME	SUPADDRESS	SUPCITY	SUPSTATUS
21	Deliwines	240, Avenue of the Americas	New York	20
32	Best Wines	660, Market Street	San Francisco	90
37	Ad Fundum	82, Wacker Drive	Chicago	95
52	Spirits & Co.	928, Strip	Las Vegas	NULL
68	The Wine Depot	132, Montgomery Street	San Francisco	10
69	Vinos del Mundo	4, Collins Avenue	Miami	92
...				

Attributes:

- Describe properties of entities (e.g., *Name*, *ID*, *Birthdate*).
- Types include:

- **Simple vs. Composite:** Single value (*Name*) vs. multiple components (*Full Name* = First + Last).
- **Single-Valued vs. Multi-Valued:** One value per entity (*Age*) vs. multiple (*Phone Numbers*).
- **Derived Attributes:** Calculated from other attributes (e.g., *Age* from *Birthdate*).



2. Relationships in ER Modeling

Types of Relationships:

- **Binary Relationships:** Involve two entities (e.g., *Employee* works for *Department*).
- **Ternary Relationships:** Involve three entities (e.g., *Doctor* treats *Patient* in *Hospital*).

Cardinality Constraints:

- Define how many instances of an entity can be associated with another:
  - **One-to-One (1:1)**

- **One-to-Many (1:N)**
- **Many-to-Many (M:N)**

Relationship Attributes:

- Additional data describing the relationship (e.g., *Date of Hire* in an *Employment* relationship).

3. Enhanced Entity-Relationship (EER) Model

Specialization & Generalization:

- **Specialization:** Breaking down an entity into sub-entities (e.g., *Employee* → *Manager*, *Engineer*).
- **Generalization:** Combining multiple entities into a higher-level entity (e.g., *Car* and *Truck* into *Vehicle*).

Categorization:

- Allows entities to belong to multiple superclasses (e.g., *Account Holder* as *Person* or *Company*).

Aggregation:

- Treats relationships as higher-level entities (useful for modeling complex data).
- Example: A *Project* may aggregate the *Consultant* and *Task* relationships.

Designing an EER Diagram:

- Identify entities and attributes.
- Define relationships with proper cardinalities.
- Apply abstraction (specialization/generalization) where needed.

Chapter 6.3 to 6.4 - SQL: Data Definition & Manipulation

1. Data Definition Language (DDL)

Key Concepts:

- Used to define, modify, and delete database structures.

Core Commands:

- `CREATE`: To create tables, views, indexes.
- `ALTER`: Modify existing tables (add columns, change data types).
- `DROP`: Delete tables or database objects.

Constraints:

- Ensure data integrity:
  - **Primary Key:** Uniquely identifies records.
  - **Foreign Key:** Maintains referential integrity between tables.
  - **NOT NULL, UNIQUE, CHECK:** Restrict values in columns.
- **Referential Actions:**
  - `ON DELETE CASCADE`: Deletes dependent records automatically.
  - `ON UPDATE SET NULL`: Sets foreign key to NULL when the parent is updated.

2. Data Manipulation Language (DML)

Core Commands:

- `SELECT`: Retrieve data from tables.
- `INSERT`: Add new records.
- `UPDATE`: Modify existing records.
- `DELETE`: Remove records from a table.

Advanced Querying Techniques:

- **Filtering Data:** Using `WHERE` clauses (e.g., `SELECT * FROM Employees WHERE Department = 'HR'`).
- **Sorting & Grouping:**
  - `ORDER BY`: Sort results.
  - `GROUP BY`: Group data for aggregation (e.g., totals, averages).
  - `HAVING`: Apply conditions on grouped data.

Joins & Subqueries:

- **Inner Join:** Combines rows with matching values in both tables.
- **Left Join:** Includes all records from the left table, with matched records from the right.
- **Nested Subqueries:** Queries within queries for complex data retrieval.

Chapter 7.2 - Structured Query Language (SQL)

1. SQL Overview

- **Purpose:** Standard language for managing relational databases.

Key Features:

- Case-insensitive syntax (`SELECT` = `select`).
- Declarative language—focuses on *what* data is needed, not *how* to get it.

2. SQL Data Definition Language (DDL) Recap

Creating Tables:

```
CREATE TABLE Employees (
ID INT PRIMARY KEY,
```

```
Name VARCHAR(50),
Department VARCHAR(30)
);
```

- **Altering Tables:** Adding/removing columns.
- **Dropping Tables:** Completely removes the table and its data.

3. SQL Data Manipulation Language (DML) in Practice

Basic Operations:

- `INSERT INTO Employees (ID, Name) VALUES (1, 'Alice');`
- `UPDATE Employees SET Department = 'Finance' WHERE ID = 1;`
- `DELETE FROM Employees WHERE ID = 1;`

Complex Queries:

- **Aggregations:** `COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`.
- **Joins:** Combine data from multiple tables for comprehensive reports.

4. Integration with Applications

SQL & APIs:

- SQL queries are embedded in applications (e.g., Java, Python).
- Enables dynamic data retrieval and manipulation from user interfaces.

5. Best Practices in SQL

Query Optimization:

- Use indexes for faster searches.
- Avoid `SELECT *` for large tables—retrieve only needed columns.

Code Readability:

- Consistent indentation and formatting.
- Use aliases for better clarity in joins.